ICT UNIVERSITY

ICT Faculty

Department of Computer Science

Intelligent Document Processing and Automated ERP Ingestion Pipeline

Project Requirements, Architecture, UML and Implementation Plan

Student: [Student name]
Programme: B.Sc. Computer Science
Academic year: 2026–2027
Supervisor: [Supervisor name]
Institution: ICT University, Cameroon

Version 1.0 – October 2026

Document Control

Item	Description
Purpose	Define the problem, scope, requirements, system design, diagrams, data model, security, testing, and implementation plan for the project.
Intended readers	Student developer, academic supervisor, evaluators, and future maintainers.
Status	Proposed design. Architecture and performance targets must be validated during implementation.
Primary technologies	Python, FastAPI, MySQL, Flutter Web, OpenCV, PyTorch, Docker, and later Kubernetes.
Important limitation	No production ERP integration, OCR accuracy, or performance result is claimed by this design document. These must be demonstrated through testing.

Abstract

Organizations receive invoices, purchase orders, receipts, forms, and other documents as paper scans, PDFs, and images. Manual data entry is slow, repetitive, and vulnerable to transcription errors. This project proposes an Intelligent Document Processing (IDP) and Automated ERP Ingestion Pipeline that accepts documents, improves their image quality, recognizes text, extracts structured fields, checks the extracted information, routes uncertain cases to a human reviewer, and prepares approved records for an Enterprise Resource Planning (ERP) system.

The proposed solution combines a custom, trainable optical character recognition (OCR) component with explicit extraction and validation rules. Image preprocessing will use OpenCV; a text-recognition model will be developed with PyTorch using a CRNN-style network and Connectionist Temporal Classification (CTC) loss. French and English are the initial target languages. A comparison against an established OCR baseline will help evaluate whether the custom component provides measurable value. The backend will use FastAPI and MySQL, while a Flutter Web dashboard will support document upload, review, status monitoring, and administration. The application will begin as a small local system and progressively be separated into services, containerized with Docker, and prepared for Kubernetes deployment only after the service boundaries are stable.

This document specifies the proposed requirements and design. The actual OCR performance, business benefits, and integration compatibility will be established by experiments and a working prototype rather than assumed in advance.

Keywords: Intelligent Document Processing, OCR, document understanding, information extraction, human-in-the-loop, ERP integration, FastAPI, Flutter, MySQL, PyTorch.

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Aim . . . 1
- 1.4 Specific Objectives . . . 2
- 1.5 Research and Engineering Questions . . . 2
- 1.6 Expected Contribution . . . 2

2 Scope, Users and Assumptions **4**
- 2.1 Initial Scope . . . 4
- 2.2 Out of Scope for the First Prototype . . . 4
- 2.3 Stakeholders and Roles . . . 4
- 2.4 Assumptions and Dependencies . . . 5

3 Requirements Specification **6**
- 3.1 Functional Requirements . . . 6
- 3.2 Non-Functional Requirements . . . 7
- 3.3 Business Rules . . . 8

4 Proposed System Architecture **9**
- 4.1 Architecture Overview . . . 9
- 4.2 Logical Modules . . . 9
- 4.3 Why Start with a Modular Monolith? . . . 10
- 4.4 Suggested Repository Structure . . . 11

5 End-to-End Workflow **12**
- 5.1 Document Lifecycle . . . 12
- 5.2 Activity Diagram: Processing a Document . . . 13
- 5.3 Sequence Diagram: Upload to OCR Result . . . 14
- 5.4 Use-Case Diagram . . . 14

6 OCR, Extraction and Validation Design **15**
- 6.1 Difference Between OCR and Document Understanding . . . 15

6.2 Proposed OCR Pipeline . . . 15
6.3 Custom Recognition Model . . . 16
6.3.1 Dataset Requirements . . . 16
6.3.2 Evaluation Metrics . . . 16
6.4 Field Extraction Strategy . . . 17
6.5 Validation Rules . . . 17
6.6 Confidence Policy . . . 18

7 Data Design **19**
7.1 Database Principles . . . 19
7.2 Proposed Logical Entity Relationship Diagram . . . 19
7.3 Entity Descriptions . . . 20
7.4 Data Integrity and Indexes . . . 20

8 API Design **21**
8.1 API Conventions . . . 21
8.2 Example API Response . . . 22
8.3 Error Handling . . . 22

9 Frontend Design **23**
9.1 Main Screens . . . 23
9.2 Usability Guidelines . . . 23

10 Security, Privacy and Reliability **24**
10.1 Security Controls . . . 24
10.2 Privacy and Governance . . . 24
10.3 Reliability and Recovery . . . 25

11 Technology Stack and Rationale **26**
11.1 Dependency Installation Strategy . . . 27

12 Testing and Evaluation Plan **28**
12.1 Testing Levels . . . 28
12.2 Evaluation Metrics . . . 29
12.3 Experiment Design . . . 29
12.4 Acceptance Criteria for the Prototype . . . 29

13 Implementation Roadmap **31**
13.1 Recommended First Week . . . 32

14 Deployment and Operations **33**
14.1 Local Development . . . 33
14.2 Docker Compose Stage . . . 33

14.3 Kubernetes Stage 33
14.4 Observability 33

15 Risks and Mitigation **35**

16 Conclusion **37**

A Glossary **38**

B Suggested Demonstration Script **39**

C References and Further Reading **40**

List of Figures

4.1 Logical architecture of the proposed platform. Dashed arrows represent data access or status updates. 9

5.1 Simplified document state machine. Rejected documents and integration failures should also be represented in implementation. 12
5.2 Activity flow from upload to controlled ERP submission. In production, rejection terminates the workflow without ERP submission. 13
5.3 Sequence for asynchronous document processing. 14
5.4 Primary use cases and participating roles. 14

7.1 Logical ER diagram. The diagram is a starting point; implement actual constraints and indexes based on the final schema. 19

List of Tables

Chapter 1

Introduction

1.1 Background

Business and public-sector workflows often depend on information received in semi-structured or unstructured documents. A person may need to open a PDF, find a supplier name, invoice number, date, tax amount, and total, then type those values into a business application. Repeating this process across many documents consumes staff time and can introduce errors.

OCR converts visual text into machine-readable text, but OCR alone does not solve the full problem. A practical system must also identify the document type, locate relevant fields, normalize values, apply business rules, preserve the source document, track corrections, and deliver approved data to the target system.

1.2 Problem Statement

The project addresses the gap between incoming digital documents and structured business records. A basic OCR tool may return text without knowing which text is the invoice number or whether the total is consistent with the tax and subtotal. A useful ingestion pipeline must connect document acquisition, recognition, extraction, validation, human review, and controlled ERP delivery into a traceable workflow.

1.3 Aim

To design and implement a secure, modular document-processing platform that extracts structured information from French and English business documents and prepares validated records for ERP ingestion.

1.4 Specific Objectives

1. Provide secure document upload and document lifecycle tracking.
2. Preprocess scanned images to improve text readability.
3. Develop and evaluate a trainable OCR model for French and English text.
4. Extract key fields from supported document types using configurable rules and/or learned methods.
5. Calculate field-level confidence and identify records that require human review.
6. Validate fields using format checks, required-field checks, duplicate detection, and arithmetic/business rules.
7. Provide a dashboard for reviewing, correcting, approving, rejecting, and tracking documents.
8. Store document metadata, extracted values, validation outcomes, and audit events in MySQL.
9. Expose controlled APIs for an ERP adapter or a sandbox ERP endpoint.
10. Package the system with Docker and document a future Kubernetes deployment path.

1.5 Research and Engineering Questions

- How much does image preprocessing improve recognition on the selected document dataset?
- How does the custom OCR model compare with a selected established OCR baseline?
- Which combination of confidence, validation rules, and human review provides a useful balance between automation and correctness?
- How can documents and extracted records be processed reliably without losing traceability?
- How can the system keep ERP-specific code isolated from document-processing logic?

1.6 Expected Contribution

The main contribution is an integrated, measurable prototype rather than an unsupported claim that a custom OCR model will outperform existing tools. The work combines a trainable recognition model, domain-specific extraction and validation, human review,

auditability, and a replaceable ERP integration adapter. The comparison experiment will show where the custom component performs well and where it needs improvement.

Chapter 2

Scope, Users and Assumptions

2.1 Initial Scope

The first version will focus on a limited, clearly defined set of business documents, preferably invoices and purchase orders. The system should accept PDF and common image formats. Initial recognition languages are French and English. The ERP stage may use a sandbox API or a clearly labelled mock adapter until the target ERP and its interface are confirmed.

2.2 Out of Scope for the First Prototype

- Fully autonomous posting of every document without human review.
- Support for every document type, language, handwriting style, or layout.
- Production connection to an ERP whose API, schema, and authentication requirements are not yet known.
- A claim of legal or accounting compliance without review against applicable requirements.
- Kubernetes operations before the containerized application is stable and tested.

2.3 Stakeholders and Roles

Role	**Main responsibility**	**Permissions / use**
Administrator	Manage users, configuration, document types, and integration settings.	Create or deactivate users, manage roles, view system status, configure thresholds.

Document operator	Upload and monitor incoming documents.	Upload documents, view processing status, inspect permitted records.
Reviewer / accountant	Check extracted fields and resolve uncertainty.	Correct fields, add review notes, approve or reject records.
Integration service account	Transfer approved records to the ERP.	Use a restricted API credential; cannot administer users or change source documents.
Supervisor / auditor	Assess operation and traceability.	Read authorized logs, metrics, and audit history.

2.4 Assumptions and Dependencies

- A labelled dataset of representative French and English documents can be obtained lawfully or created for the project.
- Training and evaluation data can be separated to avoid data leakage.
- The development machine can run Python and the required libraries. GPU use is optional for the first prototype but may be useful for model training.
- MySQL is available locally during early development.
- The target ERP and its supported integration method will be selected before implementing production posting.
- Sample documents used in testing will not contain real sensitive personal or financial data unless appropriate authorization and protection exist.

Chapter 3

Requirements Specification

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	A permitted user can sign in and sign out.	Must
FR-02	The system enforces role-based access to administrative, operational, and review functions.	Must
FR-03	A user can upload a PDF, PNG, or JPEG document subject to configured size and type limits.	Must
FR-04	The system checks file type, size, and basic file integrity before accepting a document.	Must
FR-05	The original file is stored in protected storage and linked to a unique document identifier.	Must
FR-06	The system creates a processing job and records its status and timestamps.	Must
FR-07	The preprocessing module normalizes pages and applies suitable image enhancement.	Must
FR-08	The OCR component returns recognized text and confidence information where supported.	Must
FR-09	The extraction component maps recognized content to configured fields for supported document types.	Must
FR-10	The validation component checks required fields, formats, arithmetic consistency, and duplicate indicators.	Must
FR-11	The system marks low-confidence or invalid records for human review.	Must
FR-12	An authorized reviewer can correct fields, add a reason, and approve or reject a record.	Must
FR-13	Every material correction, approval, rejection, and integration attempt is recorded in an audit trail.	Must

FR-14	Only approved records can be submitted to the ERP adapter.	Must
FR-15	The system records ERP submission status, external reference, and errors without duplicating a successful submission.	Must
FR-16	Users can search and filter documents by status, date, type, or identifier.	Should
FR-17	Administrators can configure document types, validation rules, and review thresholds.	Should
FR-18	The system exposes health endpoints and basic operational metrics.	Should
FR-19	The dashboard displays processing counts and failure/review queues.	Should
FR-20	The system supports export of approved structured data for testing if ERP connectivity is unavailable.	Could

3.2 Non-Functional Requirements

ID	**Category**	**Requirement / acceptance approach**
NFR-01	Security	Authenticate users, authorize every protected operation, keep secrets outside source code, and restrict stored files. Verify with access-control tests.
NFR-02	Privacy	Collect only necessary data, define retention, and use synthetic or authorized test documents.
NFR-03	Reliability	A failed processing step must produce a visible failure state and a recoverable record rather than silently losing the job.
NFR-04	Traceability	Every document and processing run must have stable identifiers and timestamps.
NFR-05	Maintainability	Separate API, domain logic, OCR, validation, storage, and ERP adapter responsibilities.
NFR-06	Portability	The backend and dependencies should be reproducible through documented environment setup and Docker images.

NFR-07	Performance	Measure upload, preprocessing, OCR, extraction, and end-to-end processing times on a defined test set; set targets after a baseline is measured.
NFR-08	Usability	Reviewers can locate low-confidence fields and understand validation errors without reading raw logs. Test with representative users if available.
NFR-09	Scalability	Long-running OCR must be separable from interactive API requests and later scale independently.
NFR-10	Observability	Record structured logs, health status, failure categories, and processing durations without logging passwords or full sensitive document content.

3.3 Business Rules

1. A document cannot be submitted to an ERP until it has passed required validations and been approved by an authorized reviewer, unless a separately documented policy permits a different route.

2. A low-confidence field must be visible to the reviewer; a confidence value is an indicator, not proof of correctness.

3. Duplicate detection should flag likely duplicates for review; it should not automatically delete a document.

4. ERP requests should use an idempotency key or an equivalent duplicate-prevention mechanism where supported.

5. Corrections must preserve the original extracted value and record the corrected value, actor, timestamp, and reason where required.

6. Failed integration attempts must be retryable without creating duplicate business records.

Chapter 4

Proposed System Architecture

4.1 Architecture Overview

The project will be developed incrementally. The first milestone is a working local API and database connection. OCR and document workflow are added next. Docker Compose is introduced after the individual modules work. Service boundaries are then separated where there is a clear operational or maintenance benefit. Kubernetes is a later deployment target, not a prerequisite for the first prototype.

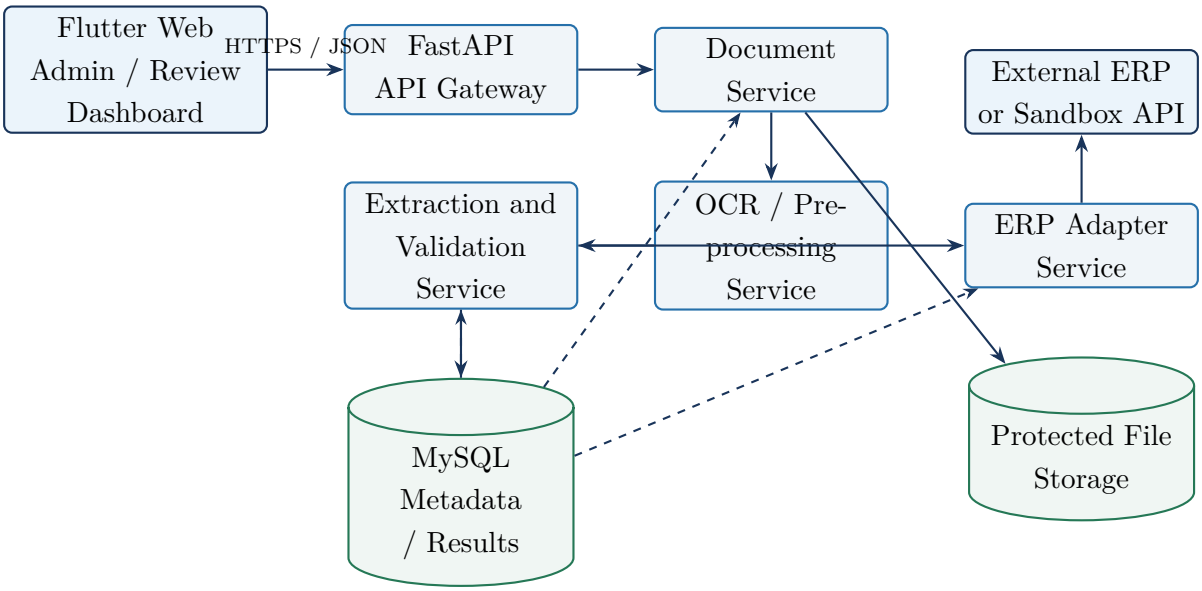


Figure 4.1: Logical architecture of the proposed platform. Dashed arrows represent data access or status updates.

4.2 Logical Modules

Module	Responsibilities	Input and output

Flutter Web UI	Login, upload, document list, review screen, approval, status dashboard.	Sends API requests; displays structured results and errors.
API / Gateway	Request validation, routing, authentication integration, consistent errors, API versioning.	HTTP requests and JSON responses.
Authentication	User identity, roles, token/session lifecycle, account status.	Credentials or identity token; authenticated principal.
Document service	Upload validation, metadata, storage reference, job lifecycle.	File and metadata; document ID and processing state.
Preprocessing / OCR	PDF page rendering, image enhancement, text recognition, OCR evaluation.	Image/page; recognized text, regions, and confidence where available.
Extraction	Convert OCR output into typed fields and preserve source evidence.	Text and layout; candidate structured fields.
Validation / review	Apply business rules, confidence policy, reviewer corrections and approvals.	Candidate fields; validation findings and approved record.
ERP adapter	Map approved records to target ERP schema, authenticate, submit, reconcile status.	Approved record; ERP response and external reference.
MySQL	Store users, metadata, extracted fields, workflow states, attempts, audit records.	Transactional structured data.
Protected file storage	Store original uploads and derived files outside the public web root.	Original files, page images, optional OCR artifacts.

4.3 Why Start with a Modular Monolith?

Microservices add network calls, deployment configuration, authentication between services, distributed failure modes, and operational overhead. For a student project, the recommended path is to implement clearly separated Python modules in one backend first. After tests confirm stable interfaces, modules can be extracted into separate services.

This preserves the intended service-oriented design without making the first milestone unnecessarily difficult.

4.4 Suggested Repository Structure

```
intelligent-document-processing/
|-- backend/
|   |-- app/
|   |   |-- main.py
|   |   |-- api/
|   |   |-- core/          # settings, security, logging
|   |   |-- models/        # SQLAlchemy models
|   |   |-- schemas/       # Pydantic request/response schemas
|   |   |-- repositories/  # database access
|   |   |-- services/
|   |   |   |-- documents/
|   |   |   |-- preprocessing/
|   |   |   |-- ocr/
|   |   |   |-- extraction/
|   |   |   |-- validation/
|   |   |   |-- integration/
|   |   |-- workers/       # background jobs, added later
|   |   |-- tests/
|   |-- alembic/
|   |-- requirements.txt
|-- frontend/flutter_app/
|-- ml/
|   |-- datasets/          # keep private; usually not committed
|   |-- notebooks/
|   |-- training/
|   |-- evaluation/
|   |-- model_artifacts/
|-- infrastructure/
|   |-- docker/
|   |-- compose/
|   |-- kubernetes/        # later stage
|-- docs/
|-- .env.example
|-- docker-compose.yml
|-- README.md
```

Chapter 5

End-to-End Workflow

5.1 Document Lifecycle

A document progresses through explicit states. The state machine should reject invalid transitions, for example, an unreviewed document must not move directly to `ERP_SENT`.

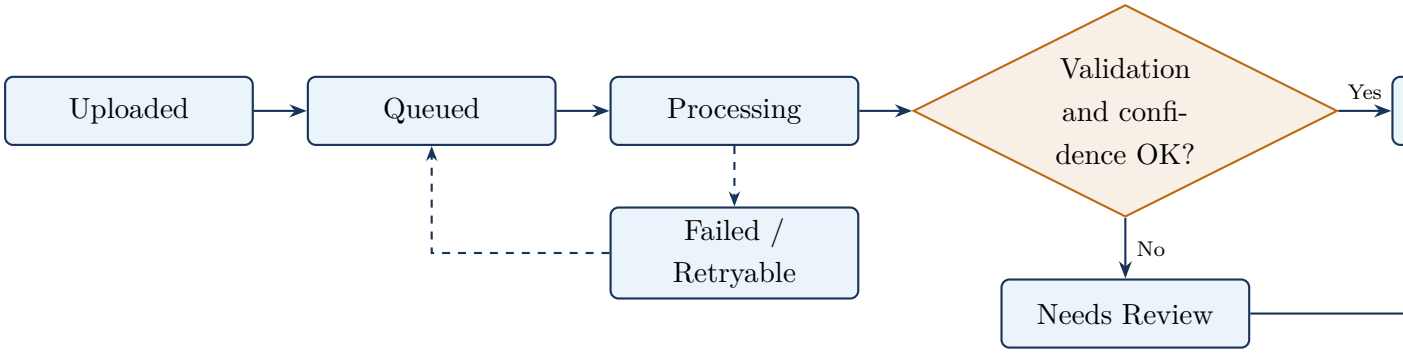


Figure 5.1: Simplified document state machine. Rejected documents and integration failures should also be represented in implementation.

5.2 Activity Diagram: Processing a Document

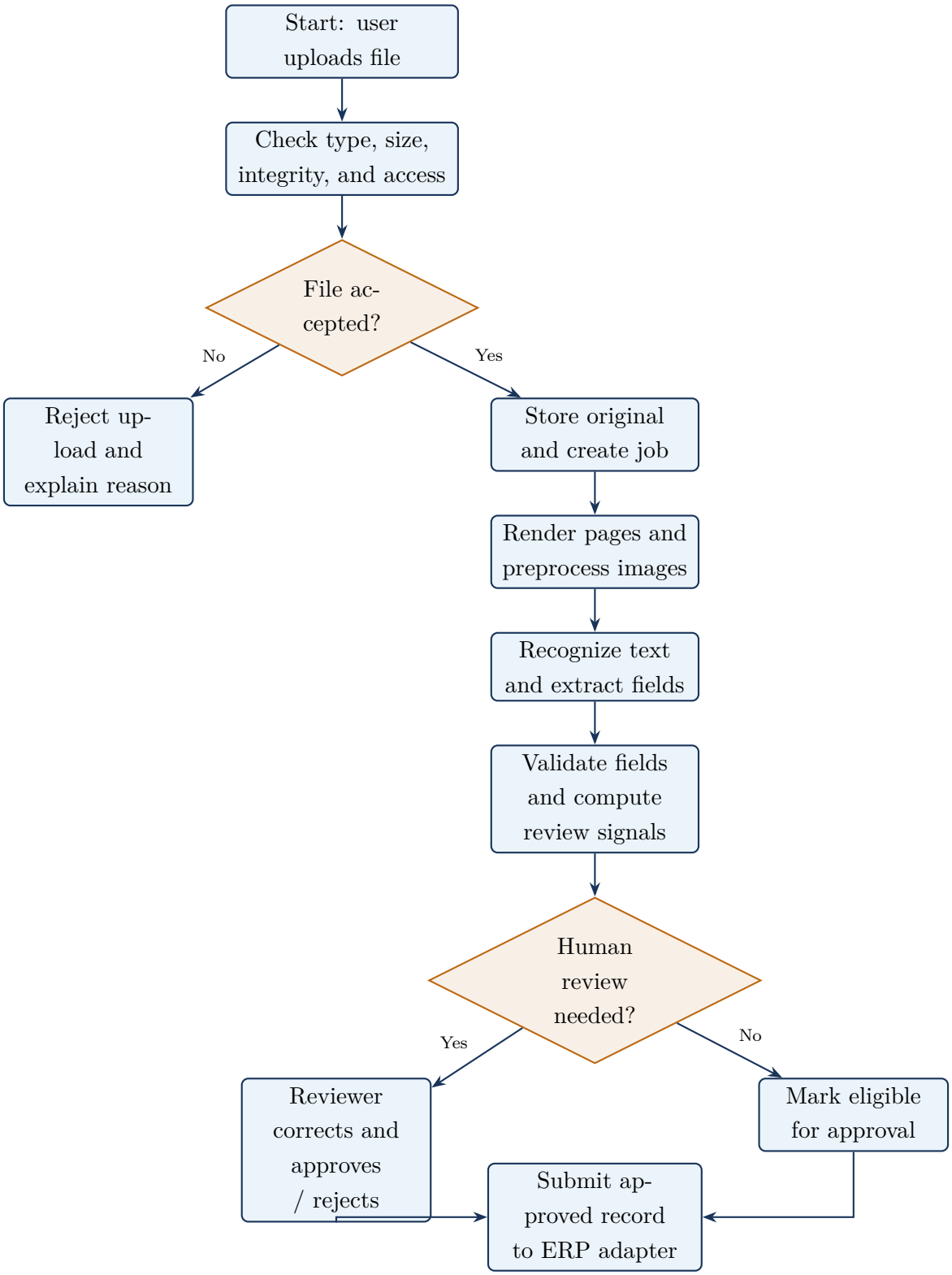


Figure 5.2: Activity flow from upload to controlled ERP submission. In production, rejection terminates the workflow without ERP submission.

5.3 Sequence Diagram: Upload to OCR Result

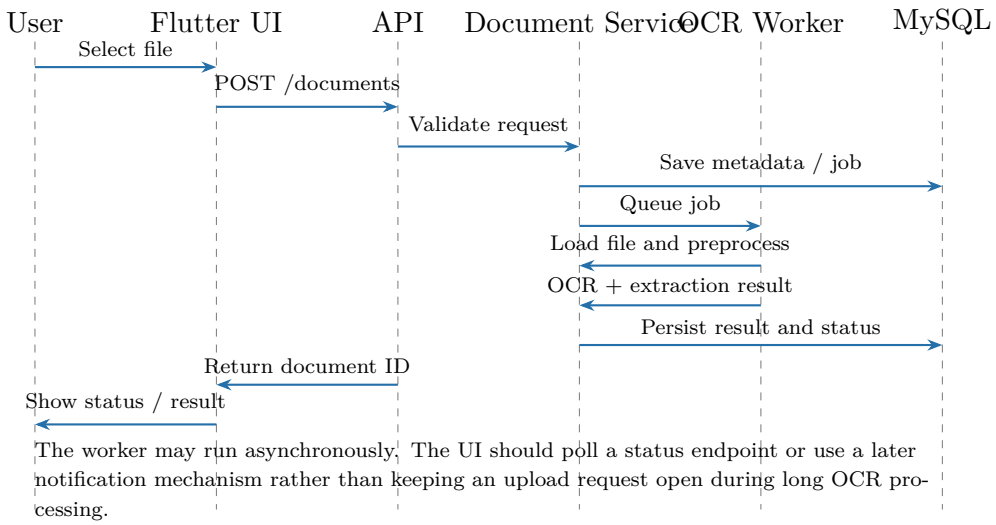


Figure 5.3: Sequence for asynchronous document processing.

5.4 Use-Case Diagram

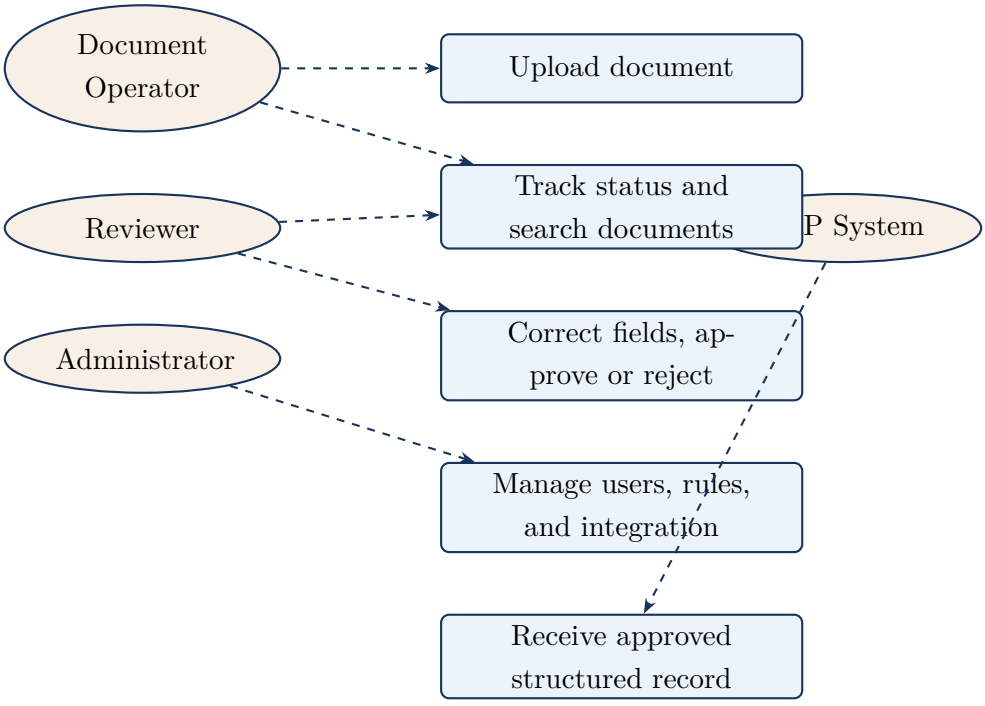


Figure 5.4: Primary use cases and participating roles.

Chapter 6

OCR, Extraction and Validation Design

6.1 Difference Between OCR and Document Understanding

OCR identifies characters or words in an image. Extraction determines which recognized values correspond to business fields. Validation checks whether the extracted values are plausible and consistent. These are separate stages so that each can be tested independently.

6.2 Proposed OCR Pipeline

1. **Input inspection:** identify file type, page count, dimensions, rotation, and basic quality.
2. **PDF rendering:** render each page to a suitable image resolution while retaining the original file.
3. **Preprocessing:** grayscale conversion when appropriate, denoising, contrast normalization, skew correction, thresholding when helpful, and resizing.
4. **Text detection / segmentation:** locate text regions or lines. The first prototype may use a constrained line-based pipeline; more complex layouts require more robust detection.
5. **Recognition:** use a trainable CRNN-style sequence recognizer with CTC loss, or a justified alternative after experiments.
6. **Post-processing:** normalize whitespace and common character confusions without silently changing source evidence.

7. **Field extraction:** combine labels, layout positions, patterns, and document-specific rules to propose values such as invoice number, supplier, dates, subtotal, tax, and total.

8. **Confidence and review:** calculate or estimate field confidence, combine it with validation outcomes, and route uncertain cases to a reviewer.

6.3 Custom Recognition Model

A CRNN-style model commonly contains a convolutional feature extractor, a sequence modelling layer such as a bidirectional recurrent network, and a character-classification output layer. CTC loss supports training when exact character-to-frame alignment is not supplied. The alphabet must cover French and English characters required by the dataset, including accented French letters, digits, punctuation, and symbols needed in business documents.

$$\mathcal{L}_{\text{CTC}} = -\log P(y \mid x)$$

where x is the input image sequence and y is the target text sequence. This equation describes the training objective conceptually; the exact implementation depends on tensor dimensions, blank-token convention, and the chosen framework.

6.3.1 Dataset Requirements

- Use representative document images and corresponding verified transcription labels.
- Keep training, validation, and test sets separate by document or source, not merely by randomly splitting nearly identical crops.
- Include clean and degraded examples: low contrast, skew, noise, different fonts, stamps, and common scan artifacts where permitted.
- Record language, document type, image dimensions, and preprocessing settings.
- Remove personal or confidential content where possible; otherwise apply authorization, access controls, and retention rules.
- Track dataset version, label corrections, model version, and evaluation results.

6.3.2 Evaluation Metrics

Character Error Rate (CER) measures character-level edits:

$$\text{CER} = \frac{S + D + I}{N}$$

where S, D, and I are character substitutions, deletions, and insertions, and N is the number of characters in the reference transcription.

Word Error Rate (WER) applies the same edit-distance idea at word level:

$$\mathrm{WER} = \frac{S_w + D_w + I_w}{N_w}.$$

Report CER and WER on the same held-out test set for both the custom recognizer and the selected baseline. Also report processing time and results by document quality or language where the dataset is large enough. A lower error rate is better; no numeric result should be written into the final report until the experiment has been run.

6.4 Field Extraction Strategy

The initial extraction approach should be deliberately bounded. For invoices, candidate fields may include:

- supplier name and supplier identifier, if present;
- invoice number;
- invoice date and due date;
- currency;
- subtotal, tax amount, and total;
- purchase order reference, where applicable.

Use regular expressions for constrained patterns such as dates and identifiers, keyword anchors such as “Invoice No.”, layout positions, and arithmetic rules. Preserve the OCR text span or page coordinates that support each field where possible. Later versions may add layout-aware machine learning once sufficient labelled data exists.

6.5 Validation Rules

Rule	**Example check**	**Outcome**
Required fields	Invoice number, supplier, date, and total must be present for the selected invoice type.	Missing field becomes a review finding.

Date parsing	Date must parse according to configured formats and permitted business range.	Invalid or ambiguous date is flagged.
Numeric parsing	Amounts are normalized using document locale conventions.	Unparseable amount is flagged.
Arithmetic consistency	Subtotal + tax and permitted adjustments approximately equal total within a configured tolerance.	Mismatch becomes a review finding, not an automatic rewrite.
Currency consistency	Currency symbols/codes agree across fields where required.	Inconsistency is flagged.
Duplicate indicator	Compare supplier, invoice number, date, amount, and optionally a file hash.	Likely duplicate is flagged for review.
ERP schema validation	Approved values satisfy required fields and enumerations of the target ERP API.	Invalid payload is blocked before submission.

6.6 Confidence Policy

Confidence must be treated as a decision signal rather than a guarantee. Establish thresholds empirically on labelled validation data. A possible policy is:

- high-confidence fields with no rule violations: eligible for expedited review or approval according to the project's chosen policy;
- medium-confidence fields: highlighted for reviewer confirmation;
- low-confidence fields or critical validation failures: mandatory human review.

For the academic prototype, human approval before ERP submission is the safer default. The report should document how thresholds were chosen and show the trade-off between automation rate and field correctness.

Chapter 7

Data Design

7.1 Database Principles

MySQL stores structured application data, not public copies of uploaded files. Original PDFs and images should live in a protected storage directory during development and in private object storage if deployed. The database stores a storage key/path, checksum, file metadata, processing status, extracted fields, and audit information. Do not store file contents in a public web directory or expose internal file paths through the API.

7.2 Proposed Logical Entity Relationship Diagram

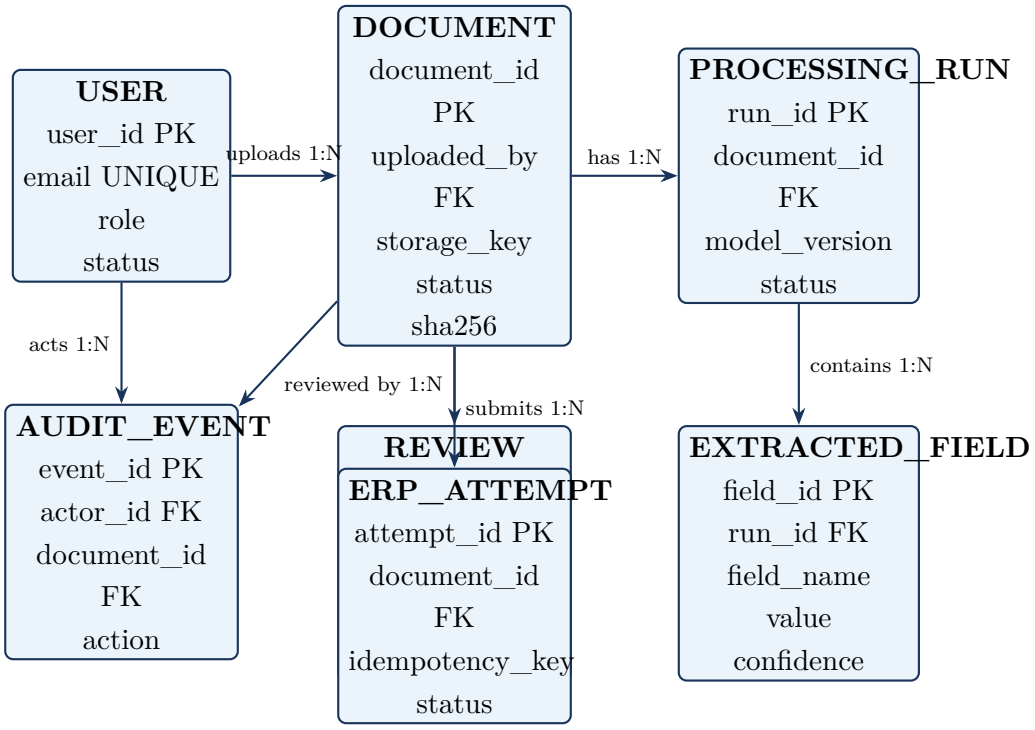


Figure 7.1: Logical ER diagram. The diagram is a starting point; implement actual constraints and indexes based on the final schema.

7.3 Entity Descriptions

Entity	Purpose and key fields
User	User identity and role. Store a password hash only if local-password authentication is selected; never store plaintext passwords.
Document	One uploaded source document, storage key, checksum, MIME type, size, uploader, creation time, and current workflow status.
ProcessingRun	One attempt to process a document, including preprocessing configuration, OCR model version, start/end time, status, and error category.
ExtractedField	A field candidate for a processing run: name, normalized value, raw value, confidence, page/region evidence, and validation status.
Review	Human review decision, reviewer, notes, timestamps, and references to before/after field values.
ERPAttempt	Each outbound integration attempt, idempotency key, destination, sanitized request/response metadata, external reference, and status.
AuditEvent	Append-oriented record of important actions such as upload, correction, approval, rejection, configuration changes, and ERP submission.

7.4 Data Integrity and Indexes

Use primary keys, foreign keys, non-null constraints for required values, unique constraints for user email, and indexes on document status, creation date, supplier/invoice identifiers when available, and processing job status. Duplicate invoice detection should be a business rule, not only a database unique constraint, because source data may be incomplete or inconsistent. Use database migrations rather than manually changing a production schema.

Chapter 8

API Design

8.1 API Conventions

Use REST-style endpoints, JSON for structured requests/responses, multipart form data for uploads, a consistent error format, and versioned routes such as `/api/v1`. Validate input with Pydantic schemas. Return a document identifier quickly and process OCR asynchronously when it may take more than a short request window.

Method	Endpoint	Purpose
POST	`/api/v1/auth/login`	Authenticate and return a session/token according to the chosen auth design.
POST	`/api/v1/documents`	Upload a new document and create a processing job.
GET	`/api/v1/documents`	List and filter documents the current user may access.
GET	`/api/v1/documents/{id}`	Get metadata, current status, and permitted processing summary.
GET	`/api/v1/documents/{id}/result`	Retrieve extracted fields, confidence, and validation findings.
GET	`/api/v1/documents/{id}/file`	Download or preview the source only after authorization checks.
PATCH	`/api/v1/documents/{id}/fields`	Save reviewer corrections with audit metadata.
POST	`/api/v1/documents/{id}/approve`	Approve a record after authorization and required checks.
POST	`/api/v1/documents/{id}/reject`	Reject a record with a reason.
POST	`/api/v1/documents/{id}/submit`	Submit an approved record to the ERP adapter.

GET	/api/v1/health	Return service health; do not expose secrets.

8.2 Example API Response

```
{
  "document_id": "doc_example_id",
  "status": "NEEDS_REVIEW",
  "document_type": "invoice",
  "fields": [
    {
      "name": "invoice_number",
      "value": "INV-2026-001",
      "confidence": 0.91,
      "validation": "PASS"
    },
    {
      "name": "total_amount",
      "value": "125000",
      "confidence": 0.68,
      "validation": "REVIEW"
    }
  ],
  "review_required": true
}
```

This is an illustrative contract, not a claim that the endpoint already exists. Production responses must be based on actual processing output and must avoid exposing another user's documents.

8.3 Error Handling

Use suitable HTTP status codes such as 400 for malformed requests, 401 for unauthenticated requests, 403 for insufficient permissions, 404 for inaccessible or missing resources, 413 for excessive upload size, 415 for unsupported media type, 409 for conflicting state transitions, and 500 for unexpected server errors. Client responses should be useful but must not expose stack traces, credentials, filesystem paths, or sensitive internal details.

Chapter 9

Frontend Design

9.1 Main Screens

1. **Sign-in:** authentication, validation feedback, and account status.
2. **Dashboard:** totals by status, recent activity, processing failures, and review queue.
3. **Upload:** drag-and-drop or file selection, accepted formats, size limits, and upload progress.
4. **Document list:** search, filtering, status, date, type, and pagination.
5. **Document detail:** source preview alongside extracted fields and validation findings.
6. **Review workspace:** editable fields, confidence indicators, source evidence, correction reason, approve/reject controls.
7. **Integration history:** submission status, attempts, sanitized errors, and external reference.
8. **Administration:** user roles, document types, validation configuration, and system settings.

9.2 Usability Guidelines

Use clear status labels, not colour alone. Keep low-confidence values visually prominent but allow users to inspect the source evidence. Show why a field failed validation. Confirm before approval or rejection. Prevent double-clicking from creating duplicate submissions. Provide loading, empty, success, and error states. The Flutter UI must not contain database credentials or ERP secrets.

Chapter 10

Security, Privacy and Reliability

10.1 Security Controls

- Enforce authentication and role-based authorization at the backend, not only in the UI.
- Hash passwords with a modern password-hashing algorithm if local passwords are used.
- Store API keys, database passwords, signing keys, and ERP credentials in environment-specific secret storage; commit only placeholders in `.env.example`.
- Restrict file extensions, MIME types, file size, and PDF page count; inspect uploads safely and avoid trusting client-supplied MIME types alone.
- Store files outside the public static directory and serve them only after an authorization check.
- Use parameterized database access through the ORM; validate and normalize inputs.
- Apply rate limits and upload quotas when exposed beyond a trusted local network.
- Use HTTPS in deployed environments and restrict CORS to known frontend origins.
- Redact tokens, passwords, full document text, and sensitive values from logs.
- Back up the database and file store together and periodically test restoration.

10.2 Privacy and Governance

Documents may contain personal, financial, or confidential information. Define who can access them, why they are retained, how long they remain stored, and how deletion is handled. Use synthetic or anonymized samples for demonstrations whenever possible. Do not send documents to external OCR or AI services unless the data handling implications and authorization are understood.

10.3 Reliability and Recovery

Each processing step should have a status and error category. Retries should be bounded and should distinguish transient errors from permanent errors. Store job state so a service restart does not silently lose work. ERP submission should be idempotent where possible. A database transaction should protect related metadata changes, while file writes should include cleanup or reconciliation for partial failures.

Chapter 11

Technology Stack and Rationale

Technology	Role	Rationale / notes
Python 3.13 (recommended initial environment)	Backend and ML language	Widely used for APIs, image processing, and model training. Use a version supported by all selected dependencies; avoid installing a pre-release Python for the project.
FastAPI	Backend HTTP API	Type-aware request validation and automatic OpenAPI documentation.
Uvicorn	Local ASGI server	Runs the FastAPI application during development.
Pydantic	Input/output schemas	Validates API payloads and gives consistent data contracts.
SQLAlchemy + Alembic	ORM and schema migrations	Keeps database access organized and schema changes reproducible.
MySQL 8	Relational database	Stores metadata, extracted values, workflow state, users, and audit records.
OpenCV	Image preprocessing	Provides image transformations such as denoising, thresholding, and geometric correction.
PyTorch	Custom OCR model	Supports model training, evaluation, and inference.
Flutter Web	User interface	Provides a cross-platform dashboard with reusable widgets.
Docker / Docker Compose	Containerization and local orchestration	Reproducible development environment and multi-container integration testing.
RabbitMQ + worker library (later)	Background job queue	Decouples slow OCR work from API requests once asynchronous processing is needed.

Kubernetes (later)	Container orchestration	Deploy and scale services after the containerized system is stable.

11.1 Dependency Installation Strategy

Do not install every ML and infrastructure dependency at once. Start with a supported Python version and a small API environment, verify the health endpoint, then add the database, file upload, preprocessing, OCR, and UI in separate milestones. Pin tested package versions in a requirements or lock file. If installation fails, diagnose the Python version and the first actual error before adding compilers or unrelated packages.

Chapter 12

Testing and Evaluation Plan

12.1 Testing Levels

Test level	Examples	Evidence
Unit tests	File validation, normalization, date/amount parsing, arithmetic rules, confidence policy.	Automated test report with expected and actual outcomes.
API tests	Authentication, upload, list, result retrieval, approval, forbidden access.	HTTP status assertions and response-schema checks.
Database tests	Foreign keys, constraints, migrations, transaction rollback.	Test database results and migration history.
OCR evaluation	CER/WER for custom model and baseline on the same test set.	Dataset description, model version, metric tables.
End-to-end tests	Upload document, process, review, approve, and submit to sandbox.	Traceable run with document ID and captured state transitions.
Security tests	Wrong-role access, unsafe file types, oversized upload, secret leakage, unauthorized file retrieval.	Pass/fail checklist and documented findings.
Recovery tests	OCR failure, database interruption, retry, duplicate ERP response.	Evidence that status remains consistent and retries do not duplicate records.
Usability tests	Reviewer completes correction and approval tasks.	Observations, task completion issues, and improvements.

12.2 Evaluation Metrics

- **OCR:** CER, WER, and inference time per page.
- **Field extraction:** precision, recall, and F1 score for each field, using a clearly defined exact-match or normalized-match rule.
- **Validation:** number of true errors detected, false alarms, and missed errors on labelled test cases.
- **Workflow:** proportion of documents completed without manual correction, proportion sent to review, rejection rate, and processing failures.
- **System:** median and 95th-percentile processing time, API error rate, retry success rate, and resource use under a stated workload.
- **ERP integration:** valid payload rate, success/failure outcomes, and duplicate-prevention tests against the sandbox.

12.3 Experiment Design

Create a test set with verified ground-truth transcriptions and field values. Run the custom OCR model and baseline on exactly the same held-out examples. Report dataset size, language distribution, document types, preprocessing configuration, model versions, and hardware. Do not tune thresholds on the test set. For extraction, report field-level metrics rather than a single accuracy figure that hides weak fields. Discuss failure cases such as skewed scans, stamps, low-resolution pages, unusual layouts, and ambiguous date/number formats.

12.4 Acceptance Criteria for the Prototype

1. A permitted user can upload a supported file and receive a stable document ID.
2. The system stores the original document privately and shows a persistent processing state.
3. A supported document can pass through preprocessing, OCR, extraction, and validation.
4. A reviewer can inspect the source, correct a field, and approve or reject the record.
5. Only an approved record can reach the ERP adapter.
6. A sandbox or mock integration demonstrates success, failure, and safe retry behaviour.

7. The project reports OCR and extraction results from a held-out dataset with a baseline comparison.

8. Automated tests cover core rules and authorization boundaries.

9. A new developer can run the documented local setup from a clean environment.

Chapter 13

Implementation Roadmap

Phase	Milestone	Tasks and exit condition	Dependency
1	Project foundation	Install supported Python; create repository and virtual environment; run FastAPI health endpoint; establish coding conventions.	None
2	Database and API	Configure MySQL, SQLAlchemy, Alembic, settings, and initial user/-document models. Demonstrate migrations and CRUD tests.	Phase 1
3	Secure upload	Validate uploads, store files privately, create metadata and statuses, build list/detail endpoints.	Phase 2
4	Preprocessing	Add PDF rendering and OpenCV steps; save intermediate images for controlled debugging.	Phase 3
5	OCR baseline	Integrate a chosen baseline and build a labelled evaluation set. Record CER/WER.	Phase 4
6	Custom OCR research	Train/evaluate the CRNN/CTC recognizer, version model artifacts, compare against baseline.	Phase 5
7	Extraction and validation	Implement document-type fields, parsing, arithmetic checks, confidence policy, and duplicate flags.	Phases 5–6
8	Human review UI	Build Flutter dashboard, source preview, field editing, approval/rejection, and audit history.	Phase 7

9	ERP adapter	Implement sandbox/mock adapter, schema mapping, idempotency, retries, and integration history.	Phase 8
10	Docker Compose	Containerize tested backend, database, worker, and frontend as appropriate; add health checks and volumes.	Phases 2–9
11	Service separation	Extract stable modules into services only where useful; add a queue for background jobs if needed.	Phase 10
12	Kubernetes and final evaluation	Prepare manifests, secrets/configuration, probes, resource limits, logs, final experiments, report, and demonstration.	Phase 11

13.1 Recommended First Week

1. Confirm Python version and create a project virtual environment.
2. Run a minimal FastAPI app and open its interactive API documentation.
3. Add a `/health` endpoint and one automated test.
4. Install and connect MySQL only after the API runs.
5. Create the first database migration and a simple document metadata model.
6. Commit working changes to Git with a short README.

Chapter 14

Deployment and Operations

14.1 Local Development

Initially, run the API locally and keep uploaded files in a dedicated private data directory excluded from Git. Keep credentials in a local `.env` file that is ignored by Git. The `.env.example` file should contain only placeholder values. Use a separate development database and never experiment with destructive schema resets on important data.

14.2 Docker Compose Stage

Once local components work, Docker Compose can run the API, MySQL, and worker in separate containers. Use named volumes for persistent database data, health checks, a private network, environment variables, and non-root containers where practical. Avoid exposing MySQL to the public internet. Do not bake secrets or model datasets into images.

14.3 Kubernetes Stage

Kubernetes is optional for the early prototype and should be attempted after Docker Compose tests pass. Likely resources include Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims or external object storage, readiness/liveness probes, resource requests/limits, and Ingress with TLS. A stateful database may be managed externally rather than hosted in the cluster for a student demonstration. Do not store secrets as plain text in source control.

14.4 Observability

Use structured logs with correlation IDs such as document ID and processing run ID. Monitor queue depth, processing duration, OCR failure categories, review backlog, ERP

failure rate, and database availability. Do not log full document contents by default. Provide a health endpoint that reports service availability without disclosing credentials or internal configuration.

Chapter 15

Risks and Mitigation

Risk	Impact	Mitigation
Insufficient labelled data	Custom OCR cannot be evaluated reliably.	Limit initial document types, create a carefully labelled dataset, document limitations, and compare against a baseline.
Custom OCR underperforms	Low field quality and too much manual review.	Use baseline comparison, inspect error categories, improve data and preprocessing, and retain a replaceable OCR interface.
Python/package incompatibility	Installation and development delays.	Use a stable supported Python release, pin dependencies, install incrementally, and document environment setup.
Unclear ERP API	Integration work cannot be completed early.	Define a generic adapter interface and use a mock/sandbox while gathering ERP requirements.
Sensitive documents exposed	Privacy and reputational harm.	Use synthetic data, private storage, strict authorization, minimal logs, and documented retention.
Duplicate ERP records	Incorrect business data.	Idempotency keys, stored attempt records, reconciliation, and retry tests.
Scope becomes too large	Core workflow remains unfinished.	Begin with invoices, a limited field set, one baseline, one custom model experiment, and a mock ERP.

Microservice overhead	More configuration and distributed failures.	Start modular, separate services only after stable interfaces, and defer Kubernetes.
OCR job crashes	Documents become stuck.	Persist job status, bounded retries, failure queues, and manual retry controls.

Chapter 16

Conclusion

This project proposes a complete workflow for transforming incoming business documents into reviewed, validated, and ERP-ready records. Its value depends on more than OCR: it requires reliable file handling, field extraction, validation, human review, secure storage, audit history, and careful integration behaviour. A custom OCR model creates a meaningful research component only if it is trained on an appropriate dataset and compared fairly with a baseline.

The recommended implementation order is deliberately incremental: first establish a working API, then database and secure upload, then preprocessing and baseline OCR, then custom OCR and extraction, followed by review UI and ERP adapter. Containerization, service separation, background queues, and Kubernetes should be introduced when the simpler system is working and the additional complexity is justified.

The final academic report should replace placeholders with the actual supervisor and student details, identify the selected ERP or sandbox, include dataset documentation, add measured results, and distinguish completed features from planned features.

Appendix A

Glossary

Term	Meaning
API	Application Programming Interface; a defined way for software components to communicate.
CTC	Connectionist Temporal Classification, a loss and decoding approach used for sequence recognition when precise alignment is not given.
CER	Character Error Rate, character-level edit errors divided by reference character count.
ERP	Enterprise Resource Planning system used to manage business processes and records.
Human-in-the-loop	A workflow in which a person reviews or corrects uncertain automated results.
IDP	Intelligent Document Processing: capture, recognize, extract, validate, and route document information.
Idempotency	A property that helps repeated requests produce no duplicate effect.
OCR	Optical Character Recognition; conversion of text in images into machine-readable text.
RBAC	Role-Based Access Control; permissions are assigned through user roles.
WER	Word Error Rate, word-level edit errors divided by reference word count.

Appendix B

Suggested Demonstration Script

1. Sign in as a document operator.
2. Upload a sample invoice and show its assigned document ID and queued status.
3. Show the processing result and extracted fields.
4. Open a low-confidence field and compare it with the source document.
5. Correct the field and enter a review note.
6. Approve the record and show the audit event.
7. Submit the approved record to the sandbox ERP adapter.
8. Show the integration response and external reference.
9. Attempt to submit again and demonstrate duplicate prevention or idempotent handling.
10. Open the evaluation summary showing the custom OCR model and baseline metrics.

Appendix C

References and Further Reading

The following are official technical resources to consult during implementation. Record the exact library versions and access dates used in the final academic submission.

- FastAPI documentation: https://fastapi.tiangolo.com/
- Flutter documentation: https://docs.flutter.dev/
- MySQL documentation: https://dev.mysql.com/doc/
- OpenCV documentation: https://docs.opencv.org/
- PyTorch documentation: https://pytorch.org/docs/
- Docker documentation: https://docs.docker.com/
- Kubernetes documentation: https://kubernetes.io/docs/
- SQLAlchemy documentation: https://docs.sqlalchemy.org/
- Alembic documentation: https://alembic.sqlalchemy.org/

Final note: This is a proposed design specification. Before submission, update the cover page, confirm the institution's formatting rules, compile the LaTeX, verify each diagram, and replace all planned or illustrative statements with implementation evidence where appropriate.